



The Intel GPA main screen

```
// Display version info in a
TextView
TextView tv = new TextView(this);
tv.append("    test
major="+ippversion[0]+"
minor="+ippversion[1]);
setContentView(tv);
}
// Load the native library
static {
System.loadLibrary("HelloIPP");
}
}
```

The Metric Chart Area at the center can show up to 16 charts for each of the available metrics. You can select and drop the relevant metrics in the chart area to show the related chart for that particular metric. These metrics are grouped into CPU, GPU, OpenGL API, Power, Memory, and Device I/O categories. You can also create custom metrics by inserting the relevant functions in your application code. These custom metrics will eventually appear under the User group of the Metrics Control Panel. The State Overrides Control Panel lets you set the various override modes for performing short experiments on the application. For more details on the override modes, you can refer to the online or bundled documentation. The Control Panel provides buttons for some of the most basic functions such as CSV export, starting/pausing the metric data collection, etc. The FPS Area and Notification Area at the top perform similar functions of displaying relevant information such as FPS values, name of the application, OpenGL ES

version, status of Vsync, size of the application window, etc. An example of retrieving the extension functions and utilizing custom metrics in your application code is as follows:

```
CreateMetricGPA_t pfn-
CreateMe-   tricGPA  =
(eglCreateMetricGPA_t)eglG
etProcAddress("eglCreateMe
tricGPA");
eglSetMetricValueGPA_t
pfnSetMetricValueGPA  =
(eglSetMetricValueGPA_t)
eglGetProcAddress("eglSet
MetricValueGPA");
eglRemoveMetricGPA_t pfn
RemoveMe-   tricGPA  =
(eglRemoveMetricGPA_t)eglG
etProcAddress("eglRemoveMe
tricGPA");
```

```
// Creating a new metric
EGLint hMetric = -1;
if (pfnCreateMetricGPA) {
hMetric = pfnCreateMetricGPA("Ref
value(42.0)", "unit", "Synthetic");
}
// Setting a value for the metric
if (pfnSetMetricValueGPA) {
pfnSetMetricValueGPA(hMetric,
42.0);
}
// Removing the Metric
if (pfnRemoveMetricGPA) {
pfnRemoveMetricGPA(hMetric);
}
```

The Intel GPA Console Client tool enables automatic collection of metric data from the command line. It lets you specify the device and application name as one of its arguments and accepts the list of metrics to collect in a configuration file. You can similarly export these metrics into a CSV file. You can type in the "gpa_console_client -h" command from the command line to find more details about the syntax.

INTEL® THREADED BUILDING BLOCKS (INTEL® TBB)

The Intel TBB is a library for developing parallel functions within your application and in your C++ code. This is relevant for Android phones and tablets as multi-core mobile processors are now the norm and ubiquitous. Using Intel TBB gives you a significant performance advantage without major effort and with typically less code. There are various forms of licenses

available—the commercial binary distribution license, an open source distribution license, and a custom license—that allow you to modify or distribute the source code of Intel TBB. The latest version can be downloaded from <http://threadingbuildingblocks.org/download>. For Android, just download the Linux version. You can then use the libtbb.so and libtbbmalloc.so libraries in the lib/android folder.

Some of the basic algorithms are parallel_for, parallel_reduce, and parallel_scan, and advanced ones include parallel_while, parallel_do, parallel_pipeline and parallel_sort. Some of the advantages of using Intel TBB for parallel programming include specification of logical parallelism instead of thread-based programming, focus on performance-based threading, easier compatibility with other packages and approaches, data parallel programming, and a focus on the generic programming approach. The parallel_for function provides one of the simplest use cases for demonstrating the parallelization in Intel TBB, which makes tasks in a loop parallel.

All you need to create a class is a function called "operator," and pass the name of this class to the parallel_for function. Blocked Ranges become important here as they represent one-directional iteration spaces that are to be split into several task ranges that may or may not be parallel. A sample of one of the examples can be seen as follows, where we first define the body object, which is an STL-style function object, and then call the parallel_for function. In this example, we are going to take the Foo function that runs on each element of an array and make it parallel.

The code to do this sequentially is as follows:

```
void SerialApplyFoo( float a[],
size_t n ) {
for( size_t i=0; i!=n; ++i )
Foo(a[i]);
} However, in order to parallelize
this loop, we define this inside a
body object as follows:
#include "tbb/tbb.h"
using namespace tbb;
class ApplyFoo {
float *const my_a;
public:
void operator()( const blocked_
range<size_t>& r ) const {
float *a = my_a;
for( size_t i=r.begin(); i!=r.
end(); ++i )
Foo(a[i]);
}
```